Virtual Memory

- Virtual memory as an alternate set of memory addresses.
- Programs use these virtual addresses rather than real addresses to store instructions and data.
- When the program is actually executed, the virtual addresses are converted into real memory addresses.

History

- virtual memory was developed in approximately 1959 – 1962, at the University of Manchester for the Atlas Computer, completed in 1962.
- In 1961, Burroughs released the B5000, the first commercial computer with virtual memory.
- Before the development of the virtual memory technique, programmers in the 1940s and 1950s had to manage directly two-level storage such as main memory or ram and secondary memory in the form of hard disks or earlier, magnetic drums.
- Enlarge the address space, the set of addresses a program can utilize.
- Virtual memory might contain twice as many addresses as main memory.
- When a computer is executing many programs at the same time, Virtual memory makes the computer to share memory efficiently.
- Eliminate a restriction that a computer works in memory which is small and be limited.
- When many programs are running at the same time, by distributing each suitable memory area to each program, VM protect programs to interfere each other in each memory area.

How does it work?

- To facilitate copying virtual memory into real memory, the operating system divides virtual memory into pages, each of which contains a fixed number of addresses.
- Each page is stored on a disk until it is needed.
- When the page is needed, the operating system copies it from disk to main memory, translating the virtual addresses into real addresses.

MMU (Memory Management Unit)

- The hardware base that makes a virtual memory system possible.
- Allows software to reference physical memory by virtual addresses, quite often more than one.
- Allows software to reference physical memory by virtual addresses, quite often more than one.
- It accomplishes this through the use of page and page tables.
- Use a section of memory to translate virtual addresses into physical addresses via a series of table lookups.
- The software that handles the page fault is generally part of an operating system and the hardware that detects this situation.

Segmentation.

- Segmentation involves the relocation of variable sized segments into the physical address space.
- Generally, these segments are contiguous units, and are referred to in programs by their segment number and an offset to the requested data.
- Efficient segmentation relies on programs that are very thoughtfully written for their target system.
- Since segmentation relies on memory that is located in single large blocks, it is very possible that enough free space is available to load a new module, but cannot be utilized.

•Segmentation may also suffer from internal fragmentation if segments are not variable-sized, where memory above the segment is not used by the program but is still "reserved" for it.

Paging

•Paging provides a somewhat easier interface for programs, in that its operation tends to be more automatic and thus transparent.

•Each unit of transfer, referred to as a page, is of a fixed size and swapped by the virtual memory

manager outside of the program's control.

•Instead of utilizing a segment/offset addressing approach, as seen in segmentation, paging uses a

linear sequence of virtual addresses which are mapped to physical memory as necessary.

•Due to this addressing approach, a single program may refer to series of many non-contiguous segments.

•Although some internal fragmentation may still exist due to the fixed size of the pages, the approach virtually eliminates external fragmentation.

•A technique used by virtual memory operating systems to help ensure that the data you need is available as quickly as possible.

•The operating system copies a certain number of pages from your storage device to main memory.

•When a program needs a page that is not in maim memory, the operating system copies the required page into memory and copies another page back to the disk.

Interrupt

Introduction About program interrupt: -

When a Process is executed by the CPU and when a user Request for another Process then this will create disturbance for the Running Process. This is also called as the Interrupt.

- Interrupts can be generated by User, Some Error Conditions and also by Software's and the hardware's.
- But CPU will handle all the Interrupts very carefully because when Interrupts are generated then the CPU must handle all the Interrupts.
- Very carefully means the CPU will also Provide Response to the Various Interrupts those are generated.
- So that When an interrupt has Occurred then the CPU will handle by using the Fetch, decode and Execute Operations.
- Interrupts allow the operating system to take notice of an external event, such as a mouse click. Software interrupts, better known as exceptions, allow the OS to handle unusual events like divide-by-zero errors coming from code execution.

The sequence of events is usually like this:

1. Hardware signals an interrupt to the processor
2. The processor notices the interrupt and suspends the currently running software
3. The processor jumps to the matching interrupt handler function in the OS
4. The interrupt handler runs its course and returns from the interrupt
5. The processor resumes where it left off in the previously running software

The most important interrupt for the operating system is the timer tick interrupt. The timer tic interrupt allows the OS to periodically regain control from the currently running user process. The OS can then decide to schedule another process, return back to the same process, do housekeeping, etc. The timer tick interrupt provides the foundation for the concept of preemptive multitasking.

TYPES OF INTERRUPTS

Generally there are three types of Interrupts those are Occurred For Example

1) Internal Interrupt
2) External Interrupt.
3) Software Interrupt.

1.Internal Interrupt:

• When the hardware detects that the program is doing something wrong, it will usually generate an interrupt usually generate an interrupt.

– Arithmetic error - Invalid Instruction

– Addressing error - Hardware malfunction

– Page fault – Debugging

• A Page Fault interrupt is not the result of a program error, but it does require the operating system to get control.

The Internal Interrupts are those which are occurred due to Some Problem in the Execution for Example When a user performing any Operation which contains any Error and which contains any type of Error. So that Internal Interrupts are those which are occurred by the Some Operations or by Some Instructions and the Operations those are not Possible but a user is trying for that Operation. And The Software Interrupts are those which are made some call to the System for Example while we are Processing Some Instructions and when we want to Execute one more Application Programs.

2.External Interrupt:

• I/O devices tell the CPU that an I/O request has completed by sending an interrupt signal to the processor.

• I/O errors may also generate an interrupt.

• Most computers have a timer which interrupts the CPU every so many interrupts the CPU every so many milliseconds.

The External Interrupt occurs when any Input and Output Device request for any Operation and the CPU will Execute that instructions first For Example When a Program is executed and when we move the Mouse on the Screen then the CPU will handle this External interrupt first and after that he will resume with his operation.

3.Software interrupts:

These types if interrupts can occur only during the execution of an instruction. They can be used by a programmer to cause interrupts if need be. The primary purpose of such interrupts is to switch from user mode to supervisor mode.

A software interrupt occurs when the processor executes an INT instruction. Written in the program, typically used to invoke a system service. A processor interrupt is caused by an electrical signal on a processor pin. Typically used by devices to tell a driver that they require attention. The clock tick interrupt is very common; it wakes up the processor from a halt state and allows the scheduler to pick other work to perform.

A processor fault like access violation is triggered by the processor itself when it encounters a condition that prevents it from executing code. Typically when it tries to read or write from unmapped memory or encounters an invalid instruction.